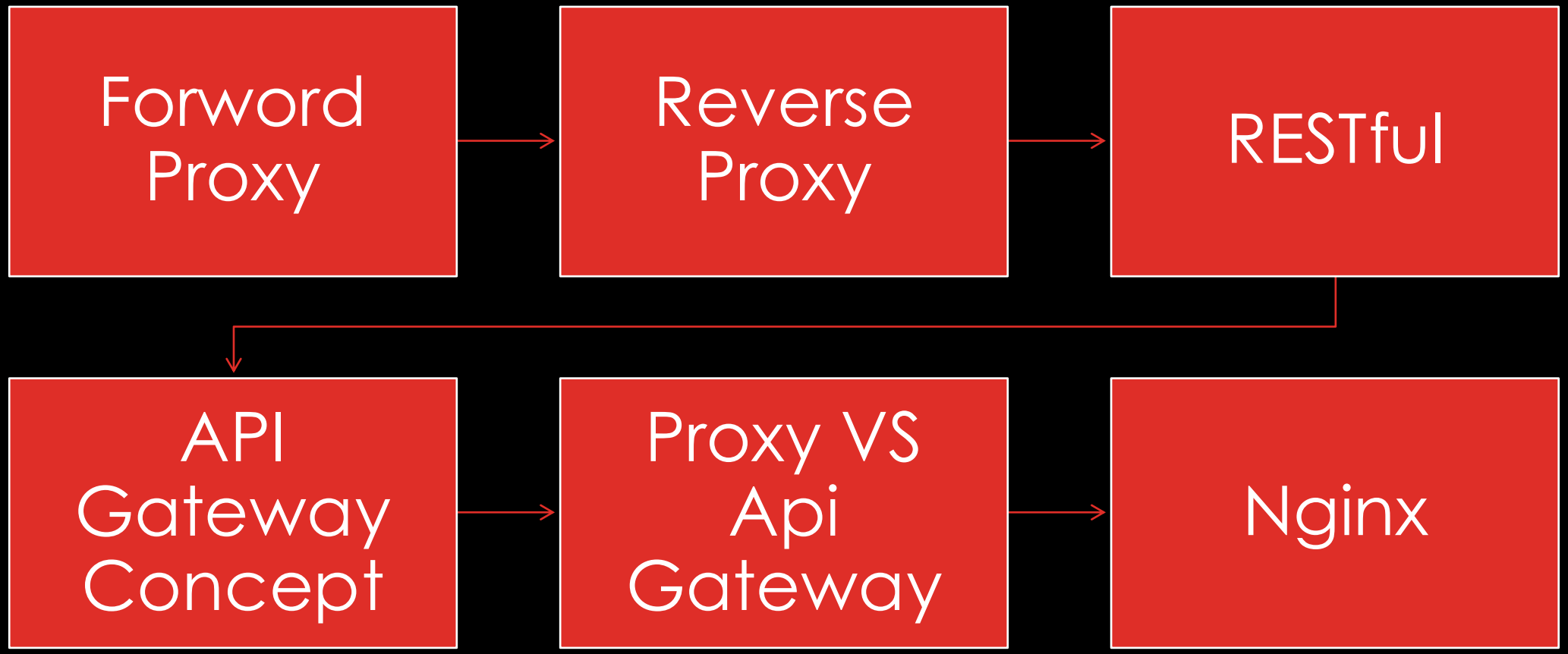
API GATEWAY

JARIN CHAROENSIRI

AGENDA


Forword Proxy
Reverse Proxy
RESTful
API Gateway Concept
Proxy VS Api Gateway
Nginx

FORWARD PROXY

FORWORD PROXY

Forward Proxy คือเซิร์ฟเวอร์ตัวกลางที่อยู่ระหว่างอุปกรณ์ของผู้ใช้ (Client) กับอินเทอร์เน็ต
เมื่อผู้ใช้ต้องการเข้าชมเว็บไซต์ใดก็ตาม คำขอของคุณจะถูกส่งไปที่ Forward Proxy ก่อน แล้ว Proxy จะเป็นคนส่งคำขอนั้นต่อไปยังเว็บไซต์ปลายทางแทนคุณ

FORWORD PROXY

หน้าที่หลัก

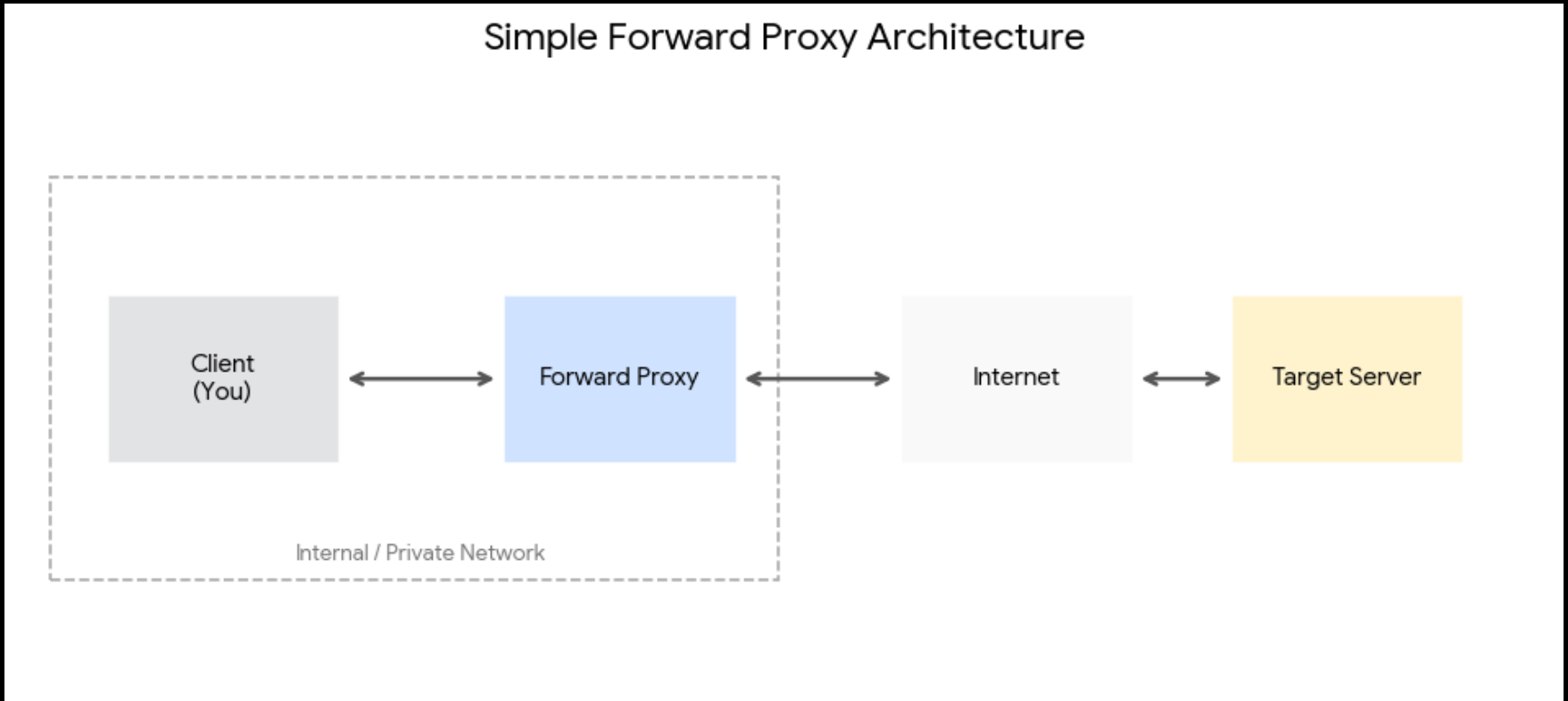
•**ปกปิดตัวตน (Anonymity):** เว็บไซต์ปลายทางจะเห็นที่อยู่ IP ของ Proxy แทนที่จะเห็น IP จริงของคุณ ช่วยรักษาความเป็นส่วนตัว

•**ควบคุมการเข้าถึง (Content Filtering):** มักใช้ในองค์กรหรือโรงเรียนเพื่อบล็อกเว็บไซต์ที่ไม่เหมาะสม หรือจำกัดการใช้งานอินเทอร์เน็ตของพนักงาน

•**ความปลอดภัย (Security):** ช่วยคัดกรองเนื้อหาที่เป็นอันตรายก่อนที่จะส่งถึงมือผู้ใช้

•**เพิ่มความเร็วด้วยการเก็บ Cache:** Proxy สามารถเก็บข้อมูลเว็บไซต์ที่ถูกเรียกใช้บ่อยๆ ไว้ในตัวเอง เมื่อมีคนอื่นต้องการดูหน้าเว็บเดิม Proxy จะส่งข้อมูลให้ทันทีโดยไม่ต้องไปโหลดใหม่จากเซิร์ฟเวอร์จริง

FORWORD PROXY



REVERSE PROXY

REVERSE PROXY

- Reverse Proxy

คือเซิร์ฟเวอร์ตัวกลางที่วางไว้ **หน้าเว็บเซิร์ฟเวอร์ (Origin Server)** เพื่อทำหน้าที่รับคำขอ (Request) จากผู้ใช้บนอินเทอร์เน็ตแทนเซิร์ฟเวอร์จริง

REVERSE PROXY

หน้าที่หลักของ Reverse Proxy

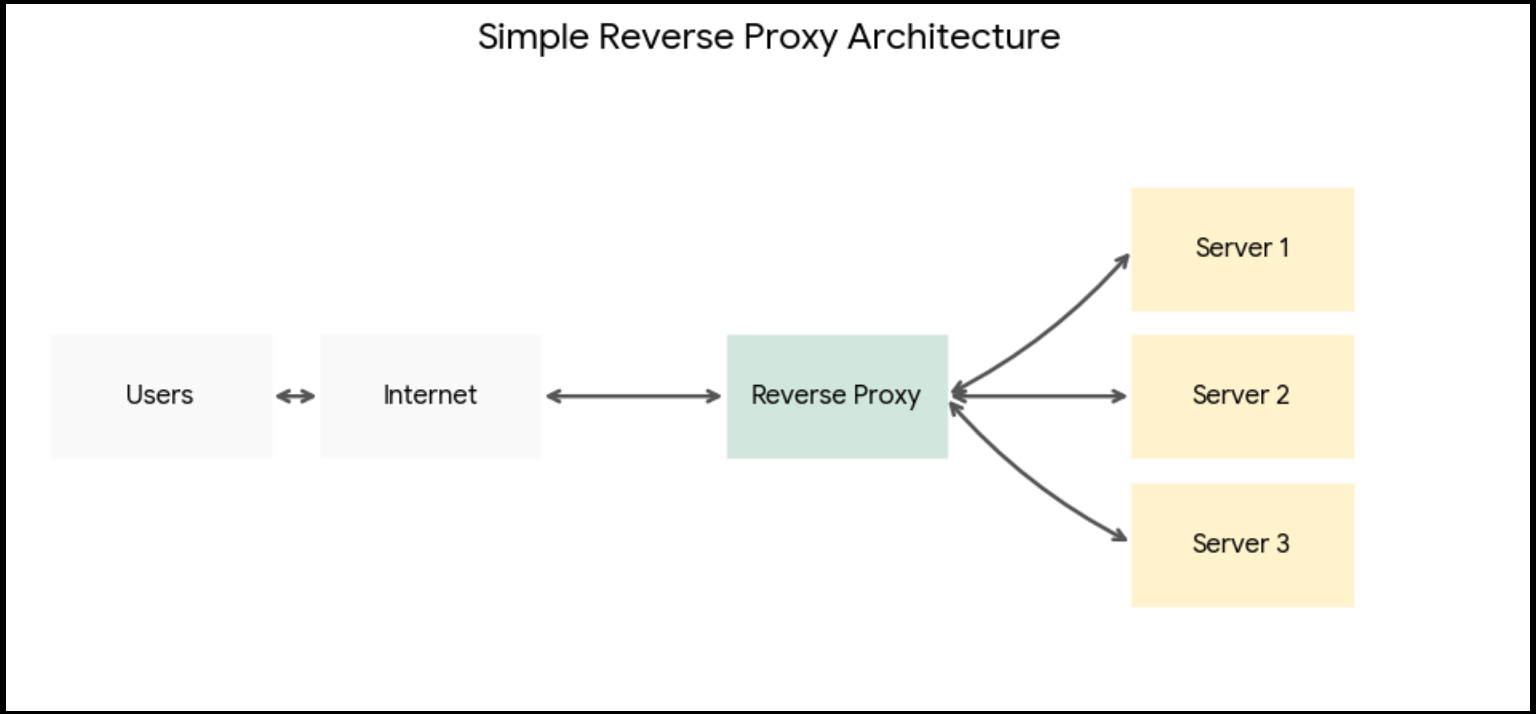
- **กระจายภาระงาน (Load Balancing):** หากคุณมีเซิร์ฟเวอร์หลายตัว Reverse Proxy จะช่วยกระจายคนเข้าชมไปยังเซิร์ฟเวอร์ที่ว่างอยู่ ไม่ให้ตัวใดตัวหนึ่งทำงานหนักเกินไป

- **ความปลอดภัย (Security):** ช่วยซ่อนตัวตนและ IP ของเซิร์ฟเวอร์จริงไว้ข้างหลัง ทำให้แฮกเกอร์โจมตีเซิร์ฟเวอร์หลักได้ยากขึ้น

- **จัดการ SSL (SSL Termination):** ทำหน้าที่ถอดรหัส HTTPS แทนเซิร์ฟเวอร์หลัก เพื่อลดภาระการประมวลผลของเซิร์ฟเวอร์ที่เก็บข้อมูลจริง

- **เพิ่มความเร็ว (Caching):** เก็บข้อมูลที่คนเรียกดูบ่อยๆ ไว้ที่ตัวมันเอง เมื่อมีคนขอข้อมูลเดิมซ้ำ มันจะส่งให้ทันทีโดยไม่ต้องไปกวนเซิร์ฟเวอร์หลัก

REVERSE PROXY



RESTFUL

RESTFUL

RESTful (ย่อมาจาก REpresentational State Transfer)

คือ **รูปแบบการออกแบบ API** (Application Programming Interface) ที่ใช้กันเป็นมาตรฐานสากล เพื่อให้ระบบคอมพิวเตอร์ 2 ระบบสามารถคุยกันได้ผ่านโปรโตคอล **HTTP**

RESTFUL

กฎกติกาการคุยกันระหว่าง
หน้าบ้าน (Frontend) กับ หลังบ้าน (Backend)

RESTFUL

หลักการทำงานสำคัญของ RESTful

Resources (ทรัพยากร): ทุกอย่างในระบบจะถูกมองเป็น "ทรัพยากร" และระบุตำแหน่งด้วย **URL** (เช่น /users, /products, /orders)

HTTP Methods (การกระทำ): ใช้คำสั่งมาตรฐานของ HTTP ในการจัดการข้อมูล ได้แก่:

- **GET**: ใช้เพื่อ "ดึงข้อมูล" (เช่น ดูรายชื่อสินค้า)
- **POST**: ใช้เพื่อ "สร้างข้อมูลใหม่" (เช่น สมัครสมาชิกใหม่)
- **PUT / PATCH**: ใช้เพื่อ "แก้ไขข้อมูล" (เช่น เปลี่ยนรหัสผ่าน)
- **DELETE**: ใช้เพื่อ "ลบข้อมูล" (เช่น ลบโพสต์)

RESTFUL

Stateless: เซิร์ฟเวอร์จะไม่เก็บสถานะ (State) ของผู้ใช้ไว้ ทุกครั้งที่ส่งคำขอไป (Request) ต้องมีข้อมูลครบถ้วนในตัวมันเองเพื่อให้เซิร์ฟเวอร์เข้าใจและทำงานได้ทันที

JSON Format: ส่วนใหญ่จะส่งข้อมูลกลับมาในรูปแบบ **JSON** ซึ่งโปรแกรมเมอร์อ่านง่ายและคอมพิวเตอร์ประมวลผลเร็ว

RESTFUL

ตัวอย่างการใช้งานจริง

แอปฯ จัดการร้านหนังสือ:

GET /books — ขอดูรายชื่อหนังสือทั้งหมด

GET /books/123 — ขอดูรายละเอียดหนังสือรหัส 123

POST /books — เพิ่มหนังสือเล่มใหม่เข้าไปในระบบ

DELETE /books/123 — ลบหนังสือเล่มนี้ทิ้ง

API GATEWAY CONCEPT

API GATEWAY

API Gateway คือ

"ประตูหน้าบ้าน" (Single Entry Point) ของระบบ **Backend** ทำหน้าที่เป็นตัวกลางคอยรับคำขอ (Request) ทั้งหมดจากฝั่งผู้ใช้งาน (Client) แล้วจัดสรรส่งต่อไปยัง Service ต่างๆ ที่อยู่ข้างหลังให้ถูกต้อง

API GATEWAY

หน้าที่หลัก

Request Routing: ดูว่าคำขอที่เข้ามาต้องการอะไร แล้วส่งต่อไปยัง Service ที่ถูกต้อง (เช่น /order ส่งไปที่ Order Service, /user ส่งไปที่ User Service)

Authentication & Authorization: ตรวจสอบบัตรผ่าน (Token/Key) ว่าผู้ใช้คนนี้มีสิทธิ์เข้าระบบไหม ก่อนจะปล่อยให้เข้าไปถึงข้อมูลข้างใน

API GATEWAY

Rate Limiting: ควบคุมปริมาณการใช้งาน ไม่ให้คนคนเดียวส่งคำขอเข้ามาถี่เกินไปจนระบบล่ม

Load Balancing: ช่วยกระจายงานไปให้เซิร์ฟเวอร์หลายๆ ตัว เพื่อไม่ให้ตัวใดตัวหนึ่งทำงานหนักเกินไป

API GATEWAY

Protocol Translation: แปลงภาษาการคุยกัน เช่น หน้าบ้านคุยแบบ HTTP/JSON แต่ข้างหลังคุยกันด้วย gRPC หรือรูปแบบอื่น

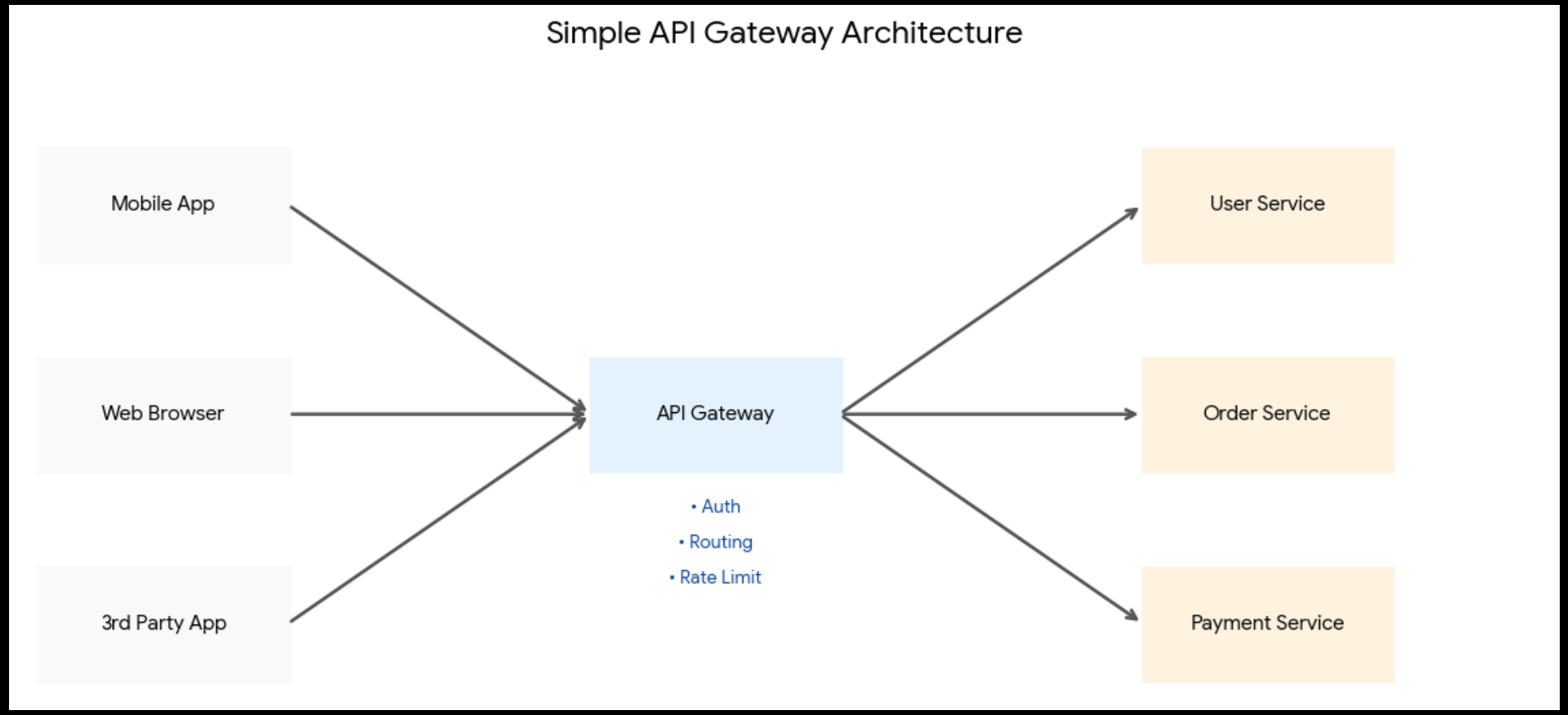
Caching: เก็บข้อมูลที่ถูกเรียกบ่อยๆ ไว้ที่ตัวเอง เพื่อส่งให้ผู้ใช้ทันทีโดยไม่ต้องไปกวน Backend

API GATEWAY

ทำไมต้องมี API Gateway?

ในระบบสมัยใหม่ที่เป็น **Microservices** (มี Service ย่อยๆ แยกกันทำงานเยอะมาก)
ถ้าไม่มี Gateway หน้าบ้านจะต้องจำ URL ของทุก Service ซึ่งยุ่งยากและไม่ปลอดภัย
การมี API Gateway ช่วยให้หน้าบ้านจำแค่ **URL เดียว** ก็เข้าถึงได้ทุกอย่าง

API GATEWAY


Simple API Gateway Architecture
Mobile App
Web Browser
3rd Party App
API Gateway
• Auth
• Routing
• Rate Limit
User Service
Order Service
Payment Service

API GATEWAY VS PROXY

API GATEWAY VS PROXY

1. Proxy (**คนขับรถส่งของ**)

•เน้นความเร็วและความเสถียร

•หน้าที่หลักคือ **"พารับและพาส่ง"** จากจุด A ไปจุด B

•**ตัวอย่าง:** ใช้ Nginx รับคนจากเว็บแล้วส่งต่อไปยัง Server หลังบ้าน 3 ตัวเพื่อกระจายโหลด (Load Balance)

API GATEWAY VS PROXY

2. API Gateway (พนักงานต้อนรับในตึกสำนักงาน)

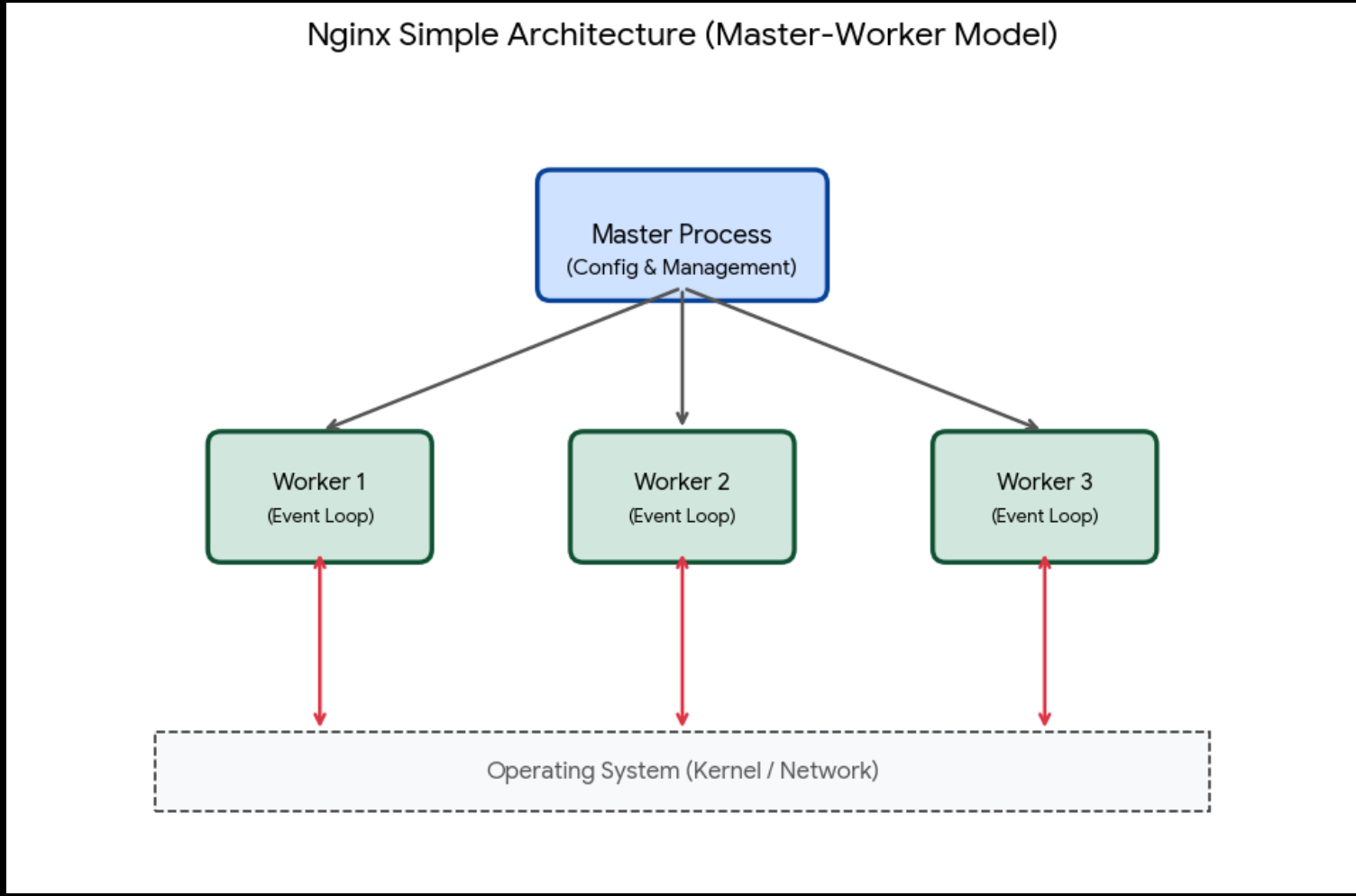
•นอกจากจะรับส่งคนแล้ว ยังต้อง **"ทำอย่างอื่นด้วย"** เช่น:

- **Authentication:** ตรวจบัตรพนักงาน (Token) ว่าเข้าแผนกนี้ได้ไหม
- **Transformation:** เปลี่ยนรูปแบบเอกสาร (เช่น รับ XML มาแต่แปลงเป็น JSON ส่งให้ข้างใน)
- **Rate Limiting:** บอกว่า "คุณเข้าพบได้แค่ชั่วโมงละ 5 ครั้งนะ"
- **Monitoring:** จดบันทึกว่าใครมาหาแผนกไหนบ่อยที่สุด เพื่อเอาไปทำสถิติหรือเก็บเงิน (Billing)

NGINX

- Architect Nginx
- Master Process
- Worker Process
- Configuration
- Use Case

NGINX


Nginx Simple Architecture (Master-Worker Model)
Master Process
(Config & Management)
Worker 1
(Event Loop)
Worker 2
(Event Loop)
Worker 3
(Event Loop)
Operating System (Kernel / Network)

NGINX

Master Process (ผู้จัดการ): ทำหน้าที่เป็นตัวหลักในการควบคุมระบบ อ่านไฟล์คอนฟิก (nginx.conf) และจัดการกระบวนการทำงานของ Worker

Worker Processes (คนทำงาน): Nginx จะสร้าง Worker ตามจำนวน CPU Core ที่มี แต่ละตัวทำงานแยกจากกันแบบ **Event Loop** (Non-blocking) ทำให้รองรับการเชื่อมต่อจำนวนมหาศาลได้พร้อมกันโดยใช้ทรัพยากรน้อย

Operating System / Network: Worker จะคุยกับระบบปฏิบัติการโดยตรงเพื่อรับส่งข้อมูลผ่านพอร์ตต่างๆ (เช่น 80 หรือ 443)

https://docs.nginx.com/nginx/admin-guide/basic-functionality/runtime-control/ - master-and-worker-processes

NGINX

- ตัวอย่าง Nginx Config

- https://github.com/tapootum/2026-k8s/tree/main/api-gateway

NGINX

- Usecase

- https://github.com/tapootum/2026-k8s/blob/main/api-gateway/usecase.md